diplomatiki

ody kon

August 2025

1 Reinforcement Learning

1.1 Introduction

Reinforcement Learning is a class of learning methods that learn to solve tasks solely by interacting with the environment. Inspired by the way humans and animals learn to perform basic tasks, in RL we try to understand how our actions lead to specific outcomes that benefit our goal, essentially through trial and error. This is the fundamental learning mechanism we observe in nature, where organisms learn to move and make decisions based only on environmental feedback. We should therefore be able to solve complex tasks without the need of prior knowledge about the problem. This is contrary to traditional problem-solving methods, such as supervised learning—where we are told what behavior is right and what is wrong—or control theory—where we have knowledge about the inner dynamics of our environment. Learning through interaction is one of the core ideas of intelligence, and cracking it would allow us to craft agents that can solve any well-defined task autonomously.

In the basic Reinforcement Learning (RL) framework, a learning agent interacts with an environment over a sequence of steps. At each step, the agent selects an action which causes the environment to transition to a new state. The agent then receives two pieces of feedback: an observation of the new state and a numerical reward signal. The agent's goal is to maximize the cumulative reward over a number of steps. This is achieved by learning a decision rule referred to as a policy that specifies which action to take for a given observation.

The description above is referred to as the agent–environment interface. The agent encompasses the learning and decision-making process, while the environment consists of everything external to the agent that it interacts with. "External" here means anything the agent cannot arbitrarily control and does not have complete knowledge of. This is different to the common notion of a physical agent i.e. a robot, and here the boundary is drawn closer to the agent. For example, in the case of a robot, certain components of its hardware may be considered part of the environment. The reward specifies the agent's objective but, since it depends on the environment's state and is not directly known to the agent, it is generally regarded as part of the environment. In contrast, the policy defines the agent's behavior and is what we adapt to achieve higher rewards,

making it a component of the agent. The state and its dynamics belong to the environment, yet in many cases the agent constructs an internal model of the world, maintaining its own beliefs about the state, dynamics, reward function, and expected returns. This internal world model is considered part of the agent.

1.2 Formal Introduction

Lets more formally define the concepts above. At each time step t the state s_t is a full desctiption of the state of the world. It fully characterizes all information meaningfull to our task about the envirnoment. In most cases the agent does not observe the complete state, it only receives an observation o_t which is a partial description of the state containing information less or equal to the state. The behaviot of the agent is defined by the policy which in general maps observations to actions a_t. The action is applied to the environment trasitioning it to a new state s_{t+1} and providing the agent a new observation o_{t+1} and a reward r_{t+1} value. The reward at each time step is calculated by the environment using a reward function $r_t = r(s_t, a_t)$. For simplicity in the following derivation we will assume $o_t = s_t$ and thus the policy is a function of the state $\pi(a_t|s_t)$.

The goal of the agent is to choose a policy π in order to maximize the expected return where the return G_t at time step t is the future sum of accumulated rewards:

$$G_t = r_t + r_{t+1} + r_{t+2} + ... + r_T \tag{1}$$

Here we have considered an episodic task with length T. The expected return starting from state s_0 under policy π is:

$$V_\pi(s = s_0) = \mathbb{E}_{a\sim\pi}[G_0|s = s_0] = \mathbb{E}_\pi[\sum_{t=0}^{T} r_t|s = s_0] \tag{2}$$

Here $V_\pi(s)$ is the value of state s under policy π defined by the expected return starting from state s and following policy π. The expectations above are under the policy π. This more formally means that for a given state produced by the environment dynamics, actions are produced according to our policy π. Following this policy produces trajectories $\tau = (s_0, a_0, s_1, a_1, .., s_{T+1})$ with probability:

$$P(\tau|s_0, \pi) = \prod_{t=0}^{T} p_{env}(s_{t+1}|s_t, a_t)\pi(a_t|s_t) \tag{3}$$

where p_{env} is the

The function V_π is the Value function and is central to solving RL problems. We can also define the value of a state-action pair $Q(s_t, a_t)$ as the expected return starting from state s_t, executing action a_t, and then following policy π.

$$Q(s = s_t, a = a_t) = \mathbb{E}_{a\sim\pi}[G_0|s = s_t, a = a_t] = \mathbb{E}_\pi[\sum_{t=0}^{T} r_t|s = s_t, a = a_t] \tag{4}$$

The expectation above are defined as under a policy. This means that the actions come from the policy π and starting from state s_0 a trajectory $\tau = (s_0, a_0, s_1, a_2, ..., s_{T-1})$ is produced. Other than the policy the produced trajectory also depends on the environment dynamics defined as transitions probabilities of states under actions $P(s_{t+1}|s_t, a_t)$ and will be more formally defined below. An expectation under a policy is the same as an expectation under the probability distribution $p(\tau)$

2 TDMPC

TD-MPC is a model-based reinforcement learning (MBRL) method that combines model predictive control (MPC) with a learned world model, policy, and value function. It leverages learned dynamics to perform model predictive path integral (MPPI) control, optimizing latent trajectories over a short-horizon reward and a terminal state value function. This allows the algorithm to incorporate both short-term and long-term returns for improved planning. TDMPC achieves performance comparable to state-of-the-art model-free reinforcement learning algorithms on a range of simulated continuous control tasks, while also exhibiting greater sample efficiency due to the use of the learned model.

2.1 Latent space planning

TD-MPC performs planning in a learned latent state representation using a variation of the Model Predictive Path Integral (MPPI) control algorithm. MPPI is a sampling-based, zeroth-order optimization method for model predictive control. Its objective is to minimize the expected cost of trajectories by iteratively refining a probability distribution over potential control sequences. The method operates by sampling a set of trajectories from the current control distribution and then updating the distribution parameters with a closed-form rule. This update is computed as a cost-weighted average of the sampled control sequences, efficiently shifting the distribution towards low-cost regions.

The version used in TD-MPC models the distribution over an action sequence of horizon H as a multivariate Gaussian distribution with diagonal covariance i.e. an independent Gaussian over each action with initial parameters $(\mu_0, \sigma_0)_{t:t+H}, \mu_0, \sigma_0 \in R^m, \mathcal{A} \in R^m$. We sample N action sequences and calculate the corresponding trajectories according to the learned dynamics model. For each trajectory Γ we estimate the total return ϕ_Γ

$$\phi_\Gamma \triangleq \mathbb{E}_\Gamma \left[\gamma^H Q_\theta(\mathbf{z}_H, \mathbf{a}_H) + \sum_{t=0}^{H-1} \gamma^t R_\theta(\mathbf{z}_t, \mathbf{a}_t) \right] \tag{5}$$

The distribution parameters are updated considering only the top-k returns ϕ_Γ^* with the following update rule:

$$\mu^j = \frac{\sum_{i=1}^k \Omega_i \Gamma_i^*}{\sum_{i=1}^k \Omega_i}, \quad \sigma^j = \sqrt{\frac{\sum_{i=1}^k \Omega_i (\Gamma_i^* - \mu^j)^2}{\sum_{i=1}^k \Omega_i}} \tag{6}$$

where Γ_i^* denotes the ith top k trajectory with return estimate ϕ_Γ^*, $\Omega_i = e^{\tau(\phi_{\Gamma,i}^*)}$ is the weight corresponding to Γ_i^* and τ is a temperature parameter that controls the sharpness of the weight.

The procedure is repeated for a fixed number of iterations J, after which we have the final optimized distribution on action sequences. We sample a trajectory from it and execute only the first action in an MPC fashion. At the next time step $t+1$, the procedure is re-initialized using the previous solution, shifting the initial mean parameter μ_0 forward by one step and repeating the planning process.

2.2 Model Components

The authors use a Task-Oriented Latent Dynamics (TOLD) model that models transitions in a latent state space, along with a reward function, a value function, and a deterministic policy, all operating in the latent space. This model is used to generate rollouts given a starting state and a number of actions, and then to estimate the value of these rollouts. The policy is used to compute optimal actions for the TD-learning targets and to generate trajectories that guide the MPPI.

The model consists of the following five learned components:

$$\text{Representation:} \quad z_t = h_\theta(s_t) \tag{7}$$

$$\text{Latent dynamics:} \quad z_{t+1} = d_\theta(z_t, a_t) \tag{8}$$

$$\text{Reward:} \quad r_t = R_\theta(z_t, a_t) \tag{9}$$

$$\text{Value:} \quad \hat{q}_t = Q_\theta(z_t, a_t) \tag{10}$$

$$\text{Policy:} \quad \hat{a}_t \sim \pi_\theta(z_t) \tag{11}$$

The components of the model are trained using transitions from a replay buffer. A given observation is encoded into a latent state z_t. The model unrolls the dynamics, and for each latent state the reward and value are predicted. For each unrolling step, a single-step loss is calculated, which is backpropagated through time up to the first encoding step. The policy is trained separately to minimize a weighted sum of Q-values.

2.3 Loss functions

All parts of the model other than the policy are trained jointly. The model is trained to minimize a temporally weigthed objective on a trajectory of fixed horizon length, sampled from a replay buffer. For each step along the trajectory, a single-step loss is computed and weighted by a discounting factor. The full objective is the following:

$$\mathcal{J} = \sum_{i=t}^{t+H} \lambda^{i-t} \mathcal{L}(\theta, -_\rangle) \tag{12}$$

Where $\Gamma \sim \mathcal{B}$ is a trajectory $(s_t, a_t, r_t, s_{t+1})_{t:t+H}$, Γ_i is the transition at step i , and $\lambda \in \mathbb{R}_+$ is a constant discount factor that prioritizes near-term predictions. The single-step loss consists of three components:

- **Latent State Consistency:** The dynamics model is trained to match the encoding of the next observation using:

$$\mathcal{L}_{\text{latent}} = \|d_\theta(z_i, a_i) - h_{\theta^-}(s_{i+1})\| \tag{13}$$

  where the parameters of the encoder are frozen for this operation.

- **Reward Loss:** The reward module is trained to match the observed rewards for a given state-action pair:

$$\mathcal{L}_{\text{reward}} = \|R_\theta(z_i, a_i) - r_i\| \tag{14}$$

- **Value Loss:** The Q-function is updated using the TD target:

$$y = R(s, a) + \gamma \max_{a'} Q_{\theta^-}(s', a') \tag{15}$$

  where Q_{θ^-} is a target network with parameters θ^- that are a moving average of the online parameters θ. The resulting value loss is:

$$\mathcal{L}_{\text{value}} = \|Q_\theta(z_i, a_i) - (r_i + \gamma Q_{\theta^-}(z_{i+1}, \pi_\theta(z_{i+1})))\| \tag{16}$$

These losses are weighted with constants $c_1 : c_3$ to form the single-step loss, which is calculated at each unrolling step. The overall loss is then backpropagated through time to update all model parameters.

Only the first observation is encoded using the "active" encoder h_θ, and the gradients of the three terms in all steps are backpropagated through time to update its parameters. In this way, the latent state space is trained to be task-oriented and predictive of reward and value, rather than being learned via a reconstruction objective that forces the network to model irrelevant details.

2.4 Policy Training

The model's policy is trained to produce actions that maximize the learned Q-function, achieved by minimizing:

$$J_\pi(\theta; \Gamma) = -\sum_{i=t}^{t+H} \lambda^{i-t}\, Q_\theta\big(z_i, \pi_\theta(sg(z_i))\big) \tag{17}$$

which sums Q-values over a rollout, weighted by a decay factor λ. This is similar to performing gradient ascent directly on the Q-function, but adapted to match the rollout scheme used in MPPI.

The learned policy is necessary to efficiently compute the value loss, which requires estimating $max_{a_t} Q_{\theta-}(a_t, z_t)$. In addition, the actor is integrated into the planning procedure: in each MPPI iteration, multiple trajectories are sampled from the policy and included in the set of candidate trajectories to be evaluated.

2.5 Replay Buffer

The model is trained in an offline manner using a Replay Buffer. We store each episode in the replay buffer and then sample trajectories from it to perform updates to the model.

3 Cross Entropy Method

3.1 Motivation

The Cross Entropy Method was originally developed as an efficient technique for estimating probabilities of rare events. Later it was found that optimization problems can be framed as rare event simulation problems and thus the CEM was adapted to solve them as well. We will motivate the main idea behind the method and the derivation of its update rule.

3.1.1 Rare event simulation

Let $\mathbf{X} = (X_1, ..., X_n)$ be a random vector that follows a probability density function $f(\cdot; \mathbf{u})$ parameterized by $\mathbf{u}$. Suppose we want to estimate the probability that the real-valued function S evaluated on a point $\mathbf{X}$ is greater than or equal to some real number γ. This probability can be expressed as

$$\ell = \mathbb{P}_{\mathbf{u}}(S(\mathbf{X}) \geq \gamma) = \mathbb{E}_{\mathbf{u}} I_{[S(\mathbf{X}) \geq \gamma]} \tag{18}$$

where the indicator I_P is 1 if P is true and 0 otherwise. If this probability is very small the event is considered rare. A standard way to estimate it is to use a simple monte carlo approximation:

$$\hat{\ell} = \frac{1}{N} \sum_{i=1}^{N} I_{S(\mathbf{X}i) \geq \gamma}, \quad \mathbf{X}i \sim f(\cdot; \mathbf{u}). \tag{19}$$

While unbiased, this estimator is highly inefficient for rare events. When sampling from the original distribution, samples that satisfy the threshold are very unlikely and thus a large amount of samples are needed to get an accurate estimate.

A common technique to reduce the variance of the estimator is importance sampling. The idea is to sample from a different distribution g where the rare event is more likely to occur, and then correct for this change of distribution by multiplying by the likelihood ratio $W(\mathbf{x}) = f(\mathbf{x}; \mathbf{u})/g(\mathbf{x})$ where f is the original distribution and g is the distribution used for sampling. The estimator is the following:

$$\hat{\ell}_{\text{IS}} = \frac{1}{N} \sum_{i=1}^{N} I_{S(\mathbf{X}i) \geq \gamma} W(\mathbf{X}i), \quad \mathbf{X}i \sim g(\cdot) \tag{20}$$

The central problem now becomes the selection of an effective g. We can see that the optimal choice for g is:

$$g^*(\mathbf{x}) = \frac{I_{S(\mathbf{x})\geq\gamma} * f(\mathbf{x};\mathbf{u})}{\ell}. \tag{21}$$

where, all of the probability mass of states which are below the threshold is discarded and dividing by ℓ normalizes the distribution. However, calculating this requires knowledge of the parameter we are trying to estimate. If we had access to this function we could use the estimator:

$$\hat{\ell} = \frac{1}{N}\sum_{i=1}^{N} I_{S(\mathbf{X}_i)\geq\gamma} W(\mathbf{X}i) = \frac{1}{N}\sum_{i=1}^{N} \ell = \ell, \tag{22}$$

which is a zero variance estimator.

The solution to this problem is to approximate the optimal g^* with a simpler distribution f from a parameterized family. This is achieved by minimizing the Kullback-Leibler (KL) divergence—also known as the cross-entropy—between g^* and f, which measures the closeness of the two distributions. The closer the parameterized distribution f is to g^*, the lower the variance of the importance sampling estimator will be. The KL divergence is:

$$\mathcal{D}_{KL}(g^*, f) = \mathbb{E}_{g^*} \ln \frac{g^*(\mathbf{X})}{f(\mathbf{X})} = \int g^*(\mathbf{x}) \ln g^*(\mathbf{x}) dx - \int g^*(\mathbf{x}) \ln f(\mathbf{x}) d\mathbf{x} \tag{23}$$

where the first term is the entropy of g^* and is a constant. We only need to minimize the second term and therefore, minimizing the KL is equivalent to solving the maximization problem:

$$\max_{\mathbf{u}} \int g^*(\mathbf{x}) \ln f(\mathbf{x};\mathbf{u}) d\mathbf{x} \tag{24}$$

Substituting g^* from (17) we get:

$$\max_{\mathbf{u}} \mathbb{E}_{\mathbf{u}} I_{S(\mathbf{X}_i)\geq\gamma} \ln f(\mathbf{X};\mathbf{u}) \tag{25}$$

Here we see that the parameter vector $\mathbf{u}$ that best approximates g^* is the one that maximizes the expected log-likelihood of the rare events under the original distribution. A key insight is that the indicator function $I_{S(X_i)\geq\gamma}$ ensures only elite samples —samples with values above the threshold— contribute to the expectation. Thus, subsequent update rules depend solely on these elite samples. This equation is also useful since for distributions belonging to the natural exponential family, it has a closed-form solution.

While this is an expectation we can estimate by sampling we run into a familiar problem: the expectation involves the rare events $S(\mathbf{X}_i) \geq \gamma$. The CEM method solves this by finding an optimal sequence of parameters $\mathbf{u}_t$ corresponding to a sequence of increasing levels γ_t iteratively. Starting from an initial parameter $\mathbf{u}_0$ and a threshold lower than the target, at each step we

find the solution to (22) using importance sampling. Our current best guess $\mathbf{w}$ (the solution of the previous step) is used as the parameter for the sampling distribution. In this way, we first solve for less rare events where γ is smaller and sample from a distribution that is iteratively improved. The new problem at each iteration, considering importance sampling is,:

$$\mathbf{u_t}^* = \underset{\mathbf{u_t}}{\operatorname{argmax}} \mathbb{E}_{\mathbf{w}} \left[I_{S(\mathbf{X}) \geq \gamma} W(\mathbf{X}; \mathbf{u_t}, \mathbf{w}) \ln f(\mathbf{X}; \mathbf{u}) \right]. \tag{26}$$

where W is the likelihood ratio and the current best guess $\mathbf{w}$ is set to be $\mathbf{u}_{t-1}$. In the algorithm the new threshold level γ_t at each step is determined by the elite samples, where we always have a fixed number k of them. When using the gaussian family of distributions the following update rule can be derived, which solves (23):

$$\mu_{t+1}^j = \frac{\sum_{i=1}^k \omega_i X_i^j}{\sum_{i=1}^k \omega_i}, \quad (\sigma_{t+1}^j)^2 = \frac{\sum_{i=1}^k \omega_i (X_i^j - \mu_{t+1}^j)^2}{\sum_{i=1}^k \omega_i} \tag{27}$$

where:

- X_i^j is the j-th component of the i-th elite sample $\mathbf{X}_i$,
- $\omega_i = W(\mathbf{X}_i; \mathbf{u_t}, \mathbf{w})$ is the likelihood ratio weight for the elite sample.

We can see that we update to the weighted mean and variance of the elite set.

3.1.2 Optimization

In the rare event simulation scenario, the CE method iteratively improves a distribution that progressively concentrates its mass in regions that produce samples with $S(\mathbf{X}_i) \geq \gamma$. This mechanism can be directly adapted to solve optimization problems.

Suppose we have our objective function $S(\mathbf{x})$ and we want to find:

$$\mathbf{x}^* = \arg\max_{\mathbf{x}} S(\mathbf{x}), \quad \gamma^* = S(\mathbf{x}^*) \tag{28}$$

We define

$$\ell(\gamma) = \mathbb{P}_{\mathbf{u}}(S(\mathbf{x}) \geq \gamma) \tag{29}$$

that is now a function of γ. As γ increases, this probability decreases as fewer and fewer points meet or exceed the threshold.

The optimization problem is now framed as a rare-event estimation problem of estimating $\ell(\gamma^*)$, only now γ^* is unknown. The CE method inherently handles this by progressively increasing the threshold γ. Applying it here will produce a sequence of distributions over $\mathbf{x}$ that become increasingly concentrated around the peaks of $S(\mathbf{x})$, thereby producing solutions to the optimization problem.

There is also another subtle, yet crucial difference in the optimization case. In the rare-event simulation scenario , the random vector $\mathbf{x}$ inherently follows a pre-defined distribution $f(\cdot; \mathbf{u})$ under which we want to calculate the rare event

probability $\mathbb{P}_{\mathbf{u}}(S(\mathbf{x}) \geq \gamma)$. In contrast, for optimization, the choice of initial $\mathbf{u}$ is entirely arbitrary. The entire distribution is a tool for searching the space.

This allows us to change the iterative process. At each step instead of finding the optimal parameters $\mathbf{u}_t$ corresponding to $\mathbb{P}_{\mathbf{u}}(S(\mathbf{x}) \geq \gamma)$, we can update to the optimal parameters corresponding to $\mathbb{P}_{\mathbf{u}_t}(S(\mathbf{x}) \geq \gamma)$, that is the probability of the rare event under the distribution from which we sample. In this way, we can omit the likelihood ratio weights ($W = 1$). For the Gaussian case, this means simply we simply update to the elite sample mean and variance:

$$\mu_{new} = \frac{1}{K}\sum_{i=1}^{k} X_i^j, \quad \sigma_{new}^2 = \frac{1}{K}\sum_{i=1}^{k}(X_i - \mu_{new})^2 \tag{30}$$

This is the standard approach to the CE method for optimization

4 Our work

We propose modifying TDMPC by using the Differentiable Cross Entropy method on a learned latent action space parameterized by the latent state space. Simillar to Amos we learn a parameterized decoder that maps pairs of latent states and actions to ground truth action sequences. This way we hope to improve the CEM sample efficiency and achieve simillar performance with fewer samples. We also aim to incorporate the state inside the DCEM making it a combination of a policy and an optimizing procedure and we try to train the induced policy π_{dCEM} using various policy gradient methods.

4.1 Latent action space DCEM

Our goal is to maximize the Expected Return

$$J(\pi) = \int_{\tau} P(\tau|\pi)R(\tau) = \mathbb{E}_{\tau\sim\pi}\big[R(\tau)\big]. \tag{31}$$

Where our return is the finite-horizon discounted return:

$$R(\tau) = \sum_{t=0}^{T-1} \gamma^t r_t. \tag{32}$$

In TDMPC our policy Π_{ϕ}^{CEM} is the CE-Method using parameters ϕ of the TOLD model, that at each step solves the optimization problem:

$$\hat{\mathbf{a}}_{\mathbf{1:H}} = \underset{\mathbf{a}_{\mathbf{1:H}}\in\mathcal{A}^{\mathcal{H}}}{\arg\max}\, G_{\phi}(\mathbf{a}_{\mathbf{1:H}}, \mathbf{x}_{\mathbf{init}}), \tag{33}$$

where G is an estimate of the expected return starting from $\mathbf{x_0}$ and taking the sequence of actions $\mathbf{a_{1:H}}$, made by the TOLD model. Specifically G_{ϕ} is

calculated as:

$$
\begin{aligned}
G_\phi(\mathbf{a_{0:H}}, \mathbf{x_{init}}) = G_\phi(\mathbf{a_{0:H}}, \mathbf{z_0}) = & \gamma^H Q_\phi(\mathbf{z_H}, \mathbf{a_H}) + \sum_{t=0}^{H-1} \gamma^t R_\phi(\mathbf{z_t}, \mathbf{a_t}), \\
\text{where: } & \mathbf{z_0} = f_\phi^{enc}(\mathbf{x_{init}}), \\
& \mathbf{z_{t+1}} = f_\phi^{d}(\mathbf{z_t})
\end{aligned}
\tag{34}
$$

where starting from initial state $\mathbf{z_0}$ governed by the dynamics f^d we wish to find the optimal sequence of actions $\hat{\mathbf{a}}_{\mathbf{1:H}}$ such that the corresponding trajectory maximizes the expected return G.

We introduce a decoder $f_\theta^{dec} : \mathcal{U} * \mathcal{Z} \rightarrow \mathcal{A}^H$ where $\mathcal{U}$ is the learned latent action space, $\mathcal{Z}$ is the latent state space of TDMPC and A is the ground truth action space. Our goal is to learn a latent action space U in which for a given state is easier to optimize in. Taking the decoder into account, the objective becomes:

$$
\hat{\mathbf{u}} = \arg\max_{\mathbf{u} \in \mathcal{U}} G_\phi(\mathbf{u}, \mathbf{z_0}) = \arg\max_{\mathbf{u} \in \mathcal{U}} G_\phi(f_\theta^{dec}(\mathbf{u}, \mathbf{z_0}), \mathbf{z_0}) \tag{35}
$$

Here, the CEM searches in the decoder's input space to find the optimal $\hat{\mathbf{u}}$ that, together with $\mathbf{z_0}$, maximizes the expected return G. Our goal is to train the decoder's parameters θ such that (35) becomes easier to solve using the Cross Entropy Method (CEM) than (37). This idea was first introduced by Amos et al. as an application of the Differentiable Cross Entropy Method. Using this differentiable variant allows us to train the action space jointly with the CEM by unrolling the optimization procedure. Consequently, the decoder can be optimized to produce better samples for the method.

The motivation for introducing such a latent space is that it enables the model to learn the structure of the action space for a specific task. This is particularly important in reinforcement learning, where tasks are often repetitive. While the CEM in its standard form already produces strong results, it employs a diagonal Gaussian distribution, thereby ignoring correlations across action dimensions and between different actions over time. The latent action space offers a way to introduce such correlations into the CEM without increasing its computational complexity.

The decoder architecture further allows us to incorporate state information into the action selection step of the CEM. In our approach, the latent action space thus becomes state-dependent. This bridges the advantages of amortizing the optimization problem—such as having a standard policy—with those of directly solving it using the Cross Entropy Method. This idea aligns with the approach taken by the authors of TDMPC, who incorporate actions produced by the policy into the action selection step of the CEM. However, we take this one step further by directly optimizing the decoder to learn the relationship between the current state and the generation of effective actions within the CEM framework.

4.1.1 DCEM with the decoder is a parameterized stochastic policy

There are several ways to interpret the entire DCEM procedure with the incorporated latent action space. One particularly useful perspective is to view it as a stochastic policy, which allows us to introduce reinforcement learning concepts in a more formal and principled manner.

The DCEM begins with a diagonal Gaussian distribution q_{μ,σ^2}, initialized with parameters $\mu = 0$ and $\sigma^2 = 2$, and iteratively refines it into an optimal distribution $q_{\hat{\mu},\hat{\sigma}}$, where $\hat{\mu}$ and $\hat{\sigma}$ denote the optimized parameters. We can thus regard the entire DCEM process as a stochastic, parameterized policy that selects latent actions by sampling from the probability distribution $q_{\hat{\mu},\hat{\sigma}}$ and decoding them using f_θ^{dec}.

We first define the policy in the latent action space as $\pi'_{\theta,\phi}(\cdot|\mathbf{z_t}, \epsilon) = q_{\hat{\mu},\hat{\sigma}}$ where ϵ represents the random seed that generates the sampling noise in DCEM. The corresponding policy in the ground-truth action space is then defined by the distribution produced at the output of the decoder, which is:

$$\pi_{\theta,\phi}(\mathbf{a_t}|\mathbf{z_t}, \epsilon) = \int \pi'_{\theta,\phi}(\mathbf{u}|\mathbf{z_t}, \epsilon) * \delta(\mathbf{a_t} - f_\theta^{dec}(\mathbf{u_t})_0)\mathbf{du} \tag{36}$$

, where $(\mathbf{x})_0$ selects the first element of $\mathbf{x}$. This integrates over all of $\mathcal{U}$ and sums the probabilities of latent action that correspond to the ground truth action $\mathbf{a_t}$. The integral is of course intractable.

4.2 Training the Decoder

4.2.1 Off policy decoder training

Our first approach is to train the decoders parameters in an off policy manner. We can train the the decoder so that the final action sequence produced by the DCEM yields a high reward in the CEM objective $G_\phi(\mathbf{a_{0:H}}, \mathbf{z_0})$ in (34):

Specifically we optimize the whole procedure $\pi_{DCEM} = \pi_{\theta,\phi}(\mathbf{a_{1:H}}|\mathbf{z}, \epsilon)$ so that the final action sequence $\hat{\mathbf{a}}_{\mathbf{1:H}} = \mu(\pi_{\theta,\phi}(\mathbf{a_{1:H}}|\mathbf{z}, \epsilon))$, where $\mu()$ is the mean, maximizes the CEM objective:

$$\hat{\mathbf{a}}_{\mathbf{1:H}} = \underset{\mathbf{a_{1:H}} \in \mathcal{A}^{\mathcal{H}}}{\arg\max}\, G_\phi(\mathbf{a_{1:H}}, \mathbf{z}) \tag{37}$$

Thus the decoders parameters are optimized according to:

$$\max_\theta \mathbb{E}_{\substack{z \sim \mathcal{D} \\ \epsilon \sim \mathcal{N}}}[G_\phi(\mu(\pi_{\theta,\phi}(\mathbf{a_{1:H}}|\mathbf{z}, \epsilon)), \mathbf{z})] \tag{38}$$

where the expectation is over latent states sampled from the replay buffer, and the gaussian noise ϵ used in the DCEM.

This objective is essentially a DDPG-style objective. Here, the policy learns only to find good solutions to the temporal difference (TD) objective and therefore learns exclusively through the reward model and Q-function, rather than directly from interactions with the environment. Consequently, the extent to which we can effectively train the decoder depends on the accuracy of the TOLD

model. Moreover, even with a well-trained model, overtraining the decoder can lead it to exploit weaknesses in the model—for instance, discovering actions that the model predicts as beneficial but that are actually suboptimal in the real environment. Such behavior can cause performance degradation or collapse.

The major advantage of this method is that it is entirely off-policy. This allows us to incorporate the decoder and train it exclusively on past experience to improve TDMPC performance, without the need to collect large amounts of new data—or any additional data at all. Moreover, applying this approach within the TDMPC framework is particularly straightforward, as the model itself is already trained in an off-policy manner.

To implement this, we backpropagate through the entire DCEM procedure. Using samples of previously visited latent states from the replay buffer, we unroll DCEM to compute the final distribution over latent actions. We then select its mean and decode it into the corresponding optimal action sequence $\hat{\mathbf{a}}_{\mathbf{0:H}}$. The associated cost $G_\phi(\hat{\mathbf{a}}_{\mathbf{1:H}}, \mathbf{z})$ is then computed, and gradients are backpropagated through all sampling stages of the DCEM. During the decoder updates, all TOLD parameters are kept frozen, as they are used in the computation of G for each sample.

For DCEM, we rely solely on samples generated by the procedure itself and do not augment them with samples from the TDMPC deterministic policy. The mean of the final distribution, $\mu(\pi'_{\theta,\phi}(\mathbf{u} \mid \mathbf{z}, \epsilon))$, is used when updating the decoder or evaluating the model. When the agent collects new data, we instead sample from the latent-space distribution $\pi'_{\theta,\phi}(\mathbf{u} \mid \mathbf{z}, \epsilon)$ to promote exploration.

DCEM UPDATE AND PLANNING CODE HERE

The rest of the model is updated in the same manner as in the standard setup. The policy is still required, as it is used to compute the target $\max_{\mathbf{a_t}} Q_\theta(\mathbf{z}, \mathbf{a_t})$, which would be computationally expensive to evaluate using the full DCEM procedure. Nevertheless, it may be worth investigating whether it is possible and beneficial to use the decoder without noise input—or DCEM with only a single iteration—for this purpose. Doing so could potentially yield a Q-function that more closely resembles the DCEM policy that is actually deployed.

To mitigate objective mismatch between the DCEM policy and the Q-function, we typically initialize the decoder such that $\Pi^{DCEM} \approx \Pi^{CEM}$. This ensures that when DCEM is introduced, the Q-function remains consistent with it. Further details on this initialization procedure and the decoder architecture are provided in the Implementation Details section.

4.2.2 On policy decoder training

We start with the general case of any stochastic, parameterized policy π_θ such as our DCEM policy. To simplify notation we consider s_i to be a general state vector, a_i to be a ground truth action and u_i to be a latent space action. The decoder from the latent to the ground truth action space is f_θ^{dec} and the corresponding decoded action applied to the environment for a latent u_i is $a_i = f_\theta^{dec}(u_i)_0$

We aim to maximize the expected return:

$$J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\big[R(\tau)\big] = \int_\tau P(\tau|\pi_\theta)R(\tau) \tag{39}$$

Where our return is the finite-horizon discounted return:

$$R(\tau) = \sum_{t=0}^{T-1} \gamma^t r_t. \tag{40}$$

The probability of a tajectory $\tau = (s_0, a_0, \ldots, s_{T+1})$ given that the environment follows dynamics $P(s_{t+1}|s_t, a_t)$ and actions follow π_θ is

$$P(\tau|\theta) = p(s_0)\prod_{t=0}^{T} P(s_{t+1}|s_t, a_t)\pi_\theta(a_t, s_t) \tag{41}$$

Substituting this into the integral expression of J and following the standard policy gradient derivation we can find the following expression for the gradient:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)R(\tau)] \tag{42}$$

This, however, is not useful for us as it requires calculating $\log \pi_\theta(a_t|s_t)$ which is the policy in the ground truth space. We would like to derive an expression consisting of $\pi'_\theta(u_t|s_t)$.

We can instead start by defining trajectories: $\tau_u = (s_0, u_0, \ldots, s_{T+1})$ consisting of the state and latent action at each step. We want to maximize the expected return following the latent policy π'_θ:

$$J(\pi_\theta) = \mathbb{E}_{\tau_{\cong}\sim\pi'_\theta}[R(\tau_u)] \tag{43}$$

Where the probability of a trajectory τ_u is

$$\begin{aligned} P(\tau|\theta) &= p(s_0)\prod_{t=0}^{T} P(s_{t+1}|s_t, a_t)\pi_\theta(a_t, s_t), \text{where } a_t = f^\theta_{dec}(u_t)_0 \\ &= p(s_0)\prod_{t=0}^{T} P(s_{t+1}|s_t, a_t)\pi'_\theta(u|z_t) * \delta(a_t - f^{dec}_\theta(u_t)_0) \\ &= p(s_0)\prod_{t=0}^{T} P(s_{t+1}|s_t, f^\theta_{dec}(u_t)_0)\pi'_\theta(u|z_t) \end{aligned} \tag{44}$$

Now we can derive:

$$\nabla_\theta J(\pi_\theta) = \nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau_u)] \tag{45}$$

$$= \nabla_\theta \int_{\tau_u} P(\tau_u|\theta)R(\tau_u) \tag{46}$$

$$= \int_{\tau_u} \nabla_\theta(P(\tau_u|\theta)R(\tau_u)) \tag{47}$$

$$= \int_{\tau_u} \nabla_\theta P(\tau_u|\theta)R(\tau_u) + \int_{\tau_u} (P(\tau_u|\theta)\nabla_\theta R(\tau_u) \tag{48}$$

$$= \int_{\tau_u} P(\tau_u|\theta)\nabla_\theta \log P(\tau_u|\theta)R(\tau_u) + \int_{\tau_u} (P(\tau_u|\theta)\nabla_\theta R(\tau_u) \tag{49}$$

$$= \mathbb{E}_{\tau_u\sim\pi_\theta}[\nabla_\theta \log P(\tau_u|\theta)R(\tau_u)] + \mathbb{E}_{\tau_u\sim\pi_\theta}[\nabla_\theta R(\tau_u)] \tag{50}$$

Where in (49) we use the log-derivative trick from calculus which gives:

$$\nabla_\theta P(\tau|\theta) = P(\tau|\theta)\nabla_\theta \log P(\tau|\theta).$$

Lets focus on the second term. When considering trajectories of the form $\tau = (s_0, a_0, ..., s_{T-1})$ we can take this term to be zero because for a given trajectory τ the return is fixed and does not depend on θ. However, this is not the case in our setting. For a single trajectory $\tau_u = (s_0, u_0, ..., s_{T-1})$ we can get many different state action trajectories τ depending on the decoder parameters θ. Consequently, given a trajectory τ_u the gradient of the return with respect to θ is not zero and the term does not cancel out.

Simillarly for the first term. If we look at the gradient:

$$\nabla_\theta \log P(\tau_u|\theta) = \cancel{\nabla_\theta \log p_0(s_0)} + \sum_{t=0}^{T} \left(\nabla_\theta \log P(s_{t+1} \mid s_t, f_\theta^{dec}(u_t)_0) + \nabla_\theta \log \pi'_\theta(u_t \mid s_t)\right) \tag{51}$$

Again, the gradient of the environment dynamics with respect to the ground truth action given a trajectory $\tau = (s_0, a_0, ..., s_{T-1})$ is zero, but this is not our case. Given the latent action u_t the probability distribution over the next state depends on the decoding $a_t = f_\theta^{dec}(u_t)_0$. Substituting (51) into (50) we get:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau_u\sim\pi_\theta}[\sum_{t=0}^{T} \left(\nabla_\theta \log P(s_{t+1} \mid s_t, f_\theta^{dec}(u_t)_0) + \nabla_\theta \log \pi'_\theta(u_t \mid s_t)\right) R(\tau_u)] + \mathbb{E}_{\tau_u\sim\pi_\theta}[\nabla_\theta R(\tau_u)] \tag{52}$$

This expression corresponds to the general case, where gradients can only be computed if a learned model of the environment is available. For TDMPC, this is possible for the second term: since we have a learned reward model, the gradient can be written as:

$$\nabla_\theta R(\tau_u) = \sum_{t=0}^{T} \gamma^t \nabla_\theta r(s_t, a_t) = \sum_{t=0}^{T} (\gamma^t \nabla_a R * \nabla_\theta f_\theta^{dec}(u_t)_0) \tag{53}$$

he first term can be simplified due to TDMPC's assumption of a deterministic dynamics model. For a trajectory $\tau_u = (s_0, u_0, .., s_{T-1})$ that is consistent with deterministic dynamics $s_{t+1} = f^{dyn}(s_t, a_t)$ where $a_t = f_\theta^{dec}(u_t)_0$ the probability $P(\tau_u|\theta)$ becomes:

$$P(\tau_u|\theta) = P(s_0) \prod_{t=0}^{T} \pi'_\theta(u_t, s_t) \tag{54}$$

and equation (55) becomes:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau_u \sim \pi_\theta}[\sum_{t=0}^{T} \nabla_\theta \log \pi'_\theta(u_t \mid s_t) R(\tau_u)] + \mathbb{E}_{\tau_u \sim \pi_\theta}[\nabla_\theta R(\tau_u)] \tag{55}$$